NO GUESSWORK LIGHTING

NGW Lab

Knowledge Base

Internal Developer Reference & FAQ

"No Guesswork Lighting"

Document Type	Internal Knowledge Base
Audience	NGW Core Engineering Team
Date	March 2026
Classification	Confidential — Internal Only

Table of Contents

Section 1: Overview & Access 3

Section 2: Workbench Tab 6

Section 3: Gold Set Tab 9

Section 4: Candidates Tab 12

Section 5: Reference Dataset Tab 14

Section 6: Common Workflows 16

Section 7: Environment & Configuration 18

Section 9: How Datasets Are Updated 20

Section 10: Lighting Intelligence — How the System Learns 25

Section 11: FAQ 29

1 Overview & Access

Section 1: Overview & Access

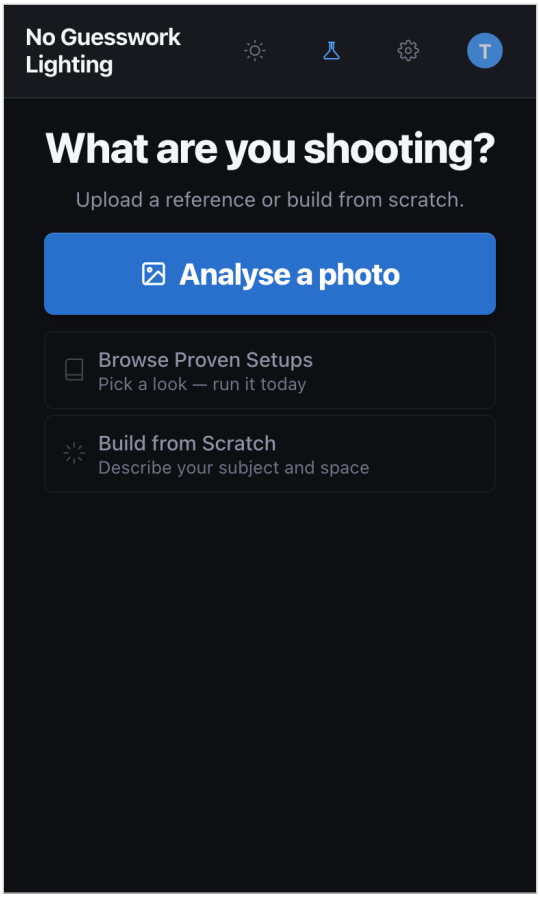
NGW Lab is the internal development environment for testing and improving the **No Guesswork Lighting** analysis engine. It is accessible only to admin-whitelisted accounts and provides four tabs: **Workbench**, **Gold Set**, **Candidates**, and **Reference Dataset**.

Purpose

- Test any portrait photo through the full CV + VLM analysis pipeline
- Maintain a benchmark truth dataset (Gold Set) for accuracy measurement
- Track proposed engine rule changes (Candidates)
- Review and curate reference images (Reference Dataset)

How to Get There

1. Sign in with an admin-whitelisted email address
2. The app loads on the Home screen
3. Look for the Lab flask icon (■) in the header — tap it to enter Lab



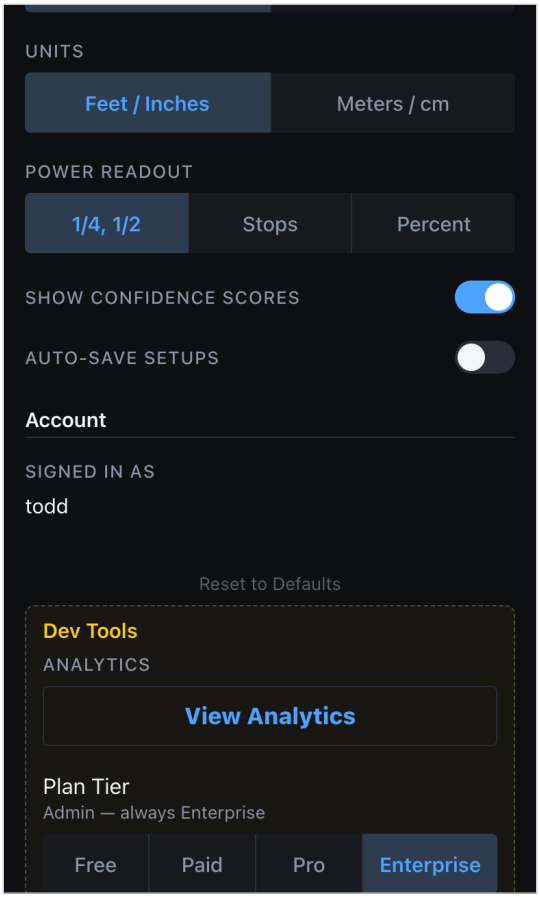
Step 3: The Lab flask icon (■) appears in the header after sign-in — tap to enter

Access Requirements

- Must be signed in with an admin-whitelisted email address
- Admin emails are defined in `auth/dev_guard.py`
- Admin accounts automatically receive enterprise-level plan access

Dev Tools — Plan Tier Override

Admin accounts always operate at Enterprise tier. In Settings → Dev Tools you can inspect your active plan tier. The selector is visible to all users but admin accounts always show **Enterprise** regardless of the chosen value.

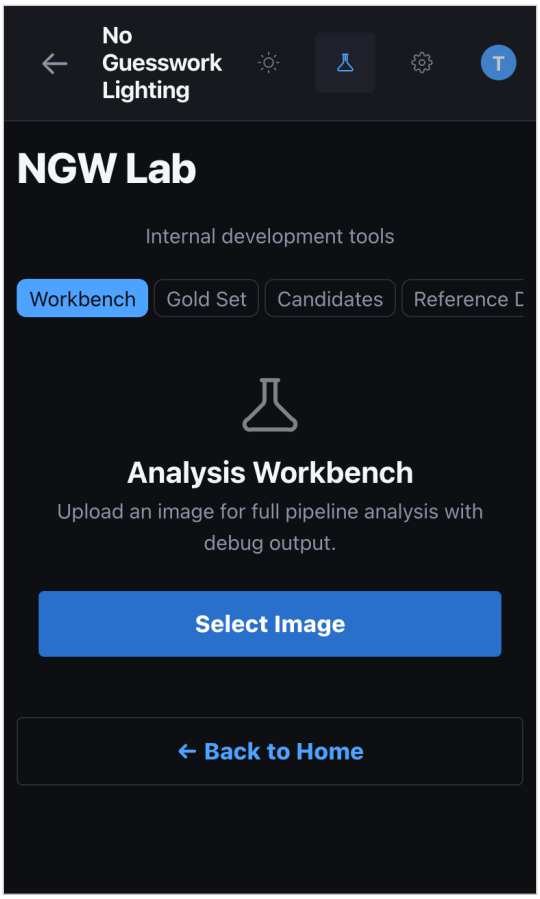


Settings → Dev Tools: admin accounts always show Enterprise tier

2 Workbench Tab

Section 2: Workbench Tab

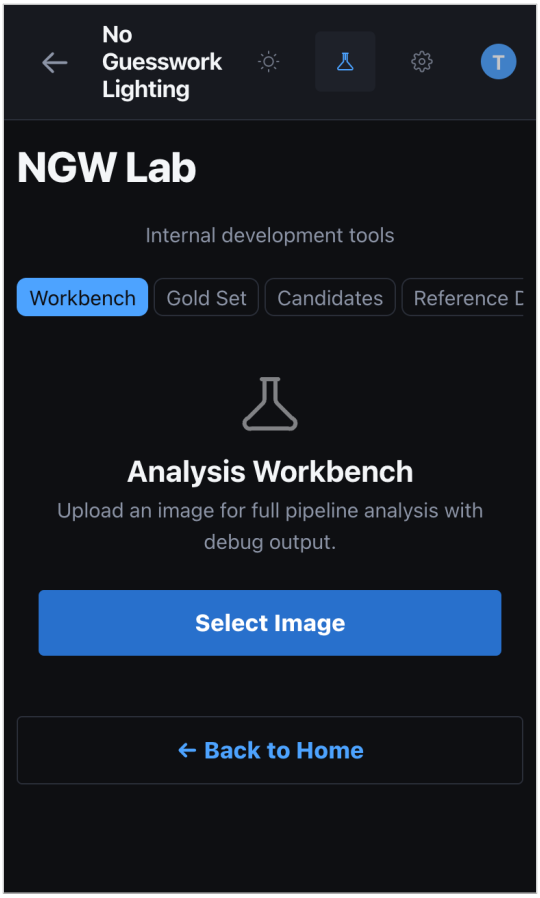
The Workbench is the primary analysis sandbox. Upload any portrait and run it through the complete NGW pipeline — computer vision passes followed by optional VLM signal extraction and physical reconstruction.



Workbench empty state — tap Select Image to begin

Step-by-Step

1. Tap the flask icon (■) in the header — lands on Workbench
2. Tap **"Select Image"** — pick a JPEG, PNG, WebP, HEIC, or TIFF (max 10 MB)
3. Optional — tick **"Debug Overlay"** before analyzing to get an annotated image
4. Tap **"Analyze"** — a scan animation plays while the pipeline runs (5–20 seconds)
5. Results appear — switch between **Formatted** / **VLM vs CV** / **Raw JSON** / **Debug Overlay** sub-tabs
6. Accept VLM overrides in the **VLM vs CV** tab if those signals are better than CV
7. Tap **"✔ Commit to Gold Set"** or **"Save to Gold Set"** — or **"Propose Rule"** for engine changes



Workbench reference — the Select Image button starts the full pipeline

Result Sub-Views

Formatted View: Human-readable cards showing Description, Narrative, Lighting (family, quality, direction, shadow, fill/rim, light count), and Recreation Setup (modifier, key placement, fill and background strategy).

VLM vs CV View: Side-by-side comparison of VLM-extracted signals versus computer vision signals. Each row has an "Accept VLM" button to override the CV value with the VLM reading. Only available when a VLM API key is configured and returned data.

Raw JSON View: The full API response as pretty-printed JSON — every signal, candidate, reliability score, and debug field.

Debug Overlay View: The annotated image with drawn overlays for shadows, highlights, catchlights, surface classes, light roles, and reconstruction geometry. Only available when "Debug Overlay" was checked before analysis.

Post-Analysis Actions

- **Save to Gold Set** — pre-fills a Gold Set entry with this image and analysis. If VLM overrides were accepted, the button turns green: "✔ Commit to Gold Set"
- **Propose Rule** — pre-fills a Candidates entry with the lighting family and setup from this analysis

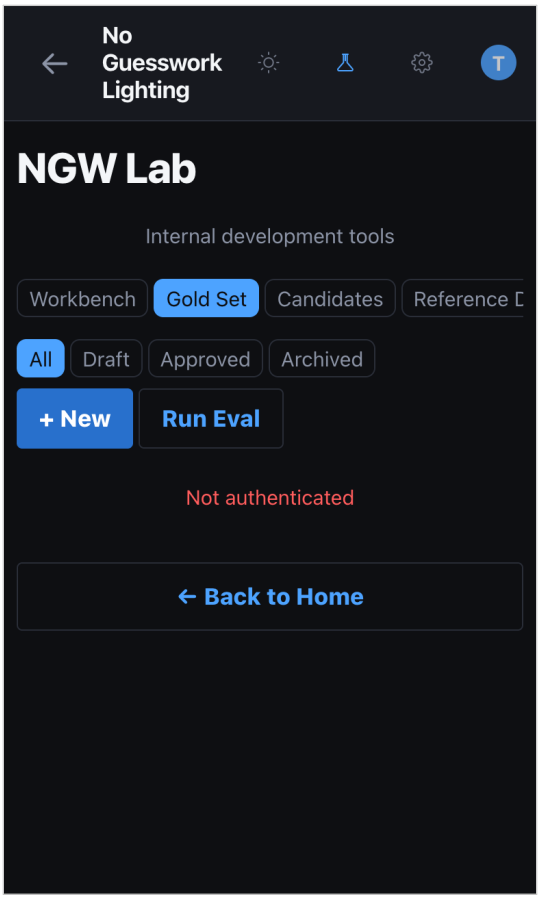
- **New Image** — clears everything and returns to the upload screen

> ***Note:*** *If the VLM vs CV tab is greyed out, either no VLM API key is set in .env, or the VLM returned no data. Check server logs for 429 rate-limit or timeout errors.*

3 Gold Set Tab

Section 3: Gold Set Tab

The Gold Set is the benchmark truth dataset. Each entry pairs a known image with the expected analysis output. It drives automated evaluation via `run_benchmarks.py` and is the ground truth for measuring engine accuracy.



Gold Set tab — status filters and action buttons

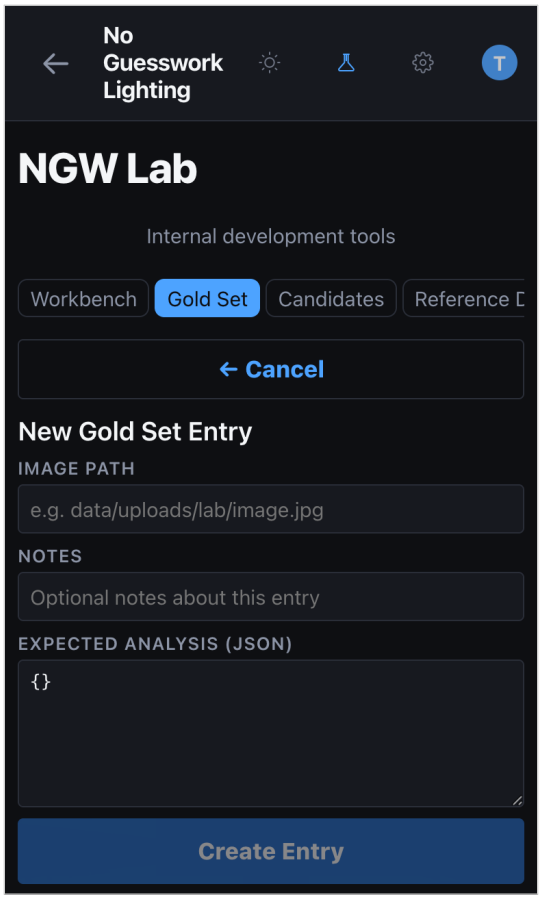
Status Values

Status	Meaning
Draft	Being created or verified — not yet eligible for benchmarks
Approved	Confirmed correct — eligible for benchmark inclusion
Archived	Retired but retained for reference — excluded from benchmarks

Adding an Entry Manually

1. Tap **"+ New"**
2. Enter the image path (relative to server working directory)

3. Paste or type the expected analysis JSON
4. Set status to **Draft** while working, **Approved** when confirmed
5. Tap **Create Entry**



New Gold Set Entry form — Image Path, Notes, and Expected Analysis JSON

Adding from Workbench (Faster)

Analyse an image in Workbench, then tap **"Save to Gold Set"**. The form auto-fills with the image path and analysis result.

Running Evaluation

Gold Set evaluation runs from the server CLI — not from the UI.

```
python3 scripts/run_benchmarks.py
```

Results print to terminal showing **PASS**, **SOFT_PASS**, and **FAIL** counts per entry.

Editing and Archiving

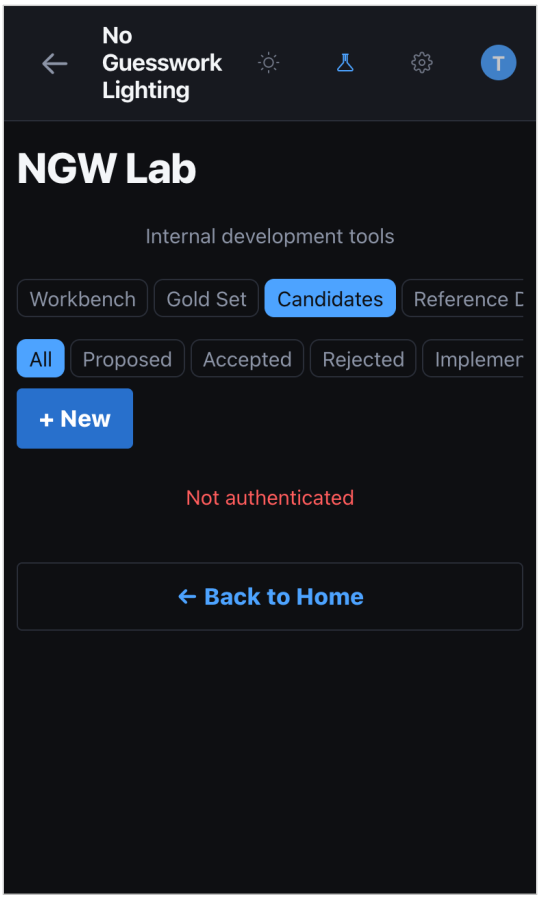
- Tap any entry to edit expected analysis, add notes, or change status

- Archive approved entries rather than deleting them — deletion is permanent

4 Candidates Tab

Section 4: Candidates Tab

The Candidates tab tracks proposed engine rule changes. When the Workbench reveals a pattern gap, contradiction, or new signal worth adding, create a Candidate to document and track it.



Candidates tab — status workflow filters

Fields

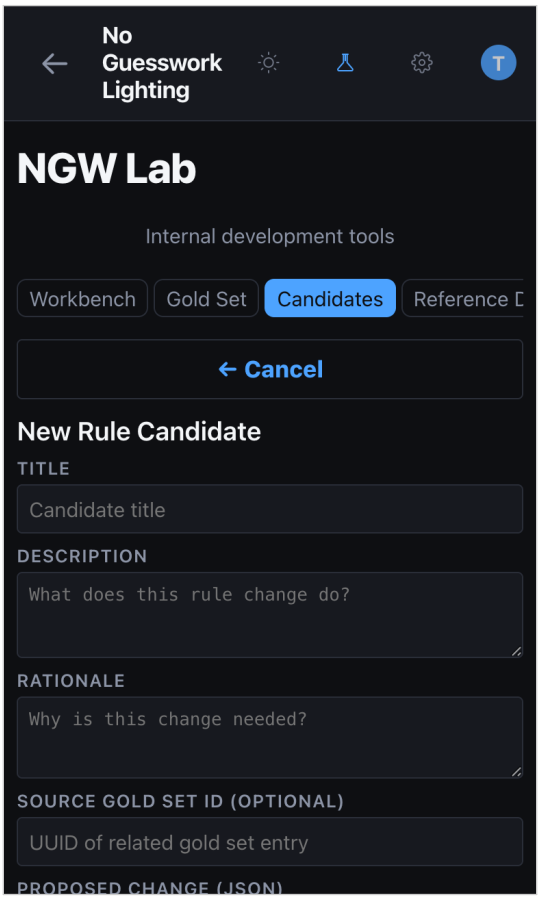
Field	Description
Title	Short description of the proposed change
Description	What the current engine gets wrong or misses
Rationale	Why this change improves accuracy, with evidence from benchmark data or Workbench results
Proposed Change	JSON describing the rule delta (pattern weights, new conditions, scoring adjustments)

Status Flow

draft → **review** → **accepted** or **rejected**

Adding a Candidate

1. Tap **"+ New"** in the Candidates tab
2. Enter a Title, Description, and Rationale for the proposed change
3. Paste the Proposed Change JSON describing the rule delta
4. Set Source Gold Set ID if the candidate was triggered by a specific benchmark entry
5. Tap **"Create Candidate"**



New Rule Candidate form — Title, Description, Rationale, and Proposed Change JSON

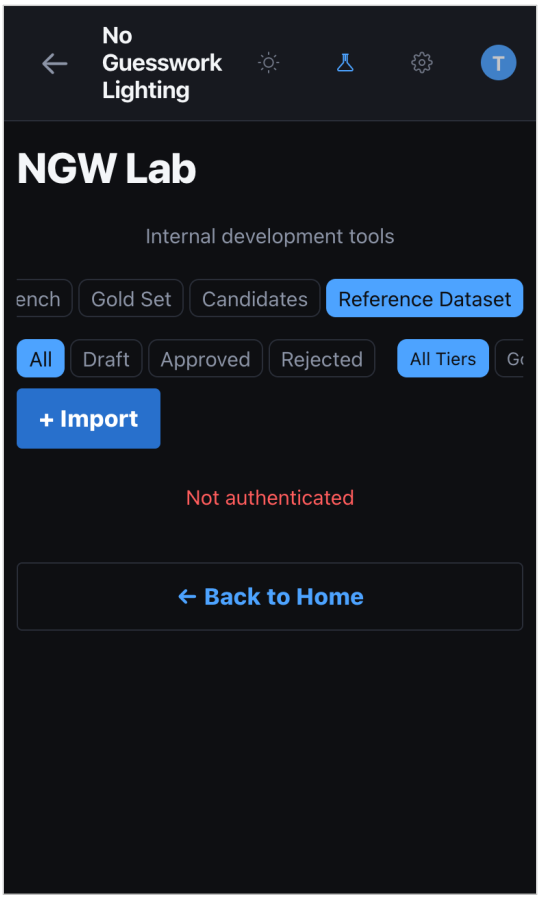
Adding from Workbench (Faster)

Tap **"Propose Rule"** after analysis. The form auto-fills with the lighting family and a reference to the source image.

5 Reference Dataset Tab

Section 5: Reference Dataset Tab

A curated library of reference photos with full pipeline analysis attached. Each entry stores the original image, VLM description, CV signals, and complete reference_analysis JSON. Used to build the pattern match library and validate the engine.



Reference Dataset — status and tier filters, + Import for new entries

Browsing

- Entries appear as a grid of authenticated thumbnails with filename and status badge
- Images load via authenticated fetch (Bearer token handled automatically)
- Tap **"Load More"** to paginate

Viewing an Entry

1. Tap any thumbnail to open the detail view
2. The full-size image renders at the top

3. Use the ← Prev / Next → arrows to navigate between entries without returning to the grid
4. The counter shows current position (e.g. "3 / 47")

Collapsible Detail Sections

- **Reference Analysis** — Core analysis JSON: lighting read, recreation setup, image read. Open by default.
- **Pipeline Signals** — Raw CV pipeline pass outputs: shadow, highlight, catchlight, geometry, etc.
- **VLM Reconstruction** — VLM-based physical reconstruction with primary candidate, alternatives, confidence, and ambiguity notes.

Actions

Action	Effect
Approve	Marks entry as a valid reference image. Eligible for benchmark inclusion.
Reject	Marks entry as unsuitable (bad framing, ambiguous lighting, duplicate). Stays in dataset but filtered from benchmarks.
Reprocess	Re-runs the full CV + VLM pipeline on the existing entry. Use after engine updates for fresh signals. Does not change stat

Ingesting New Images

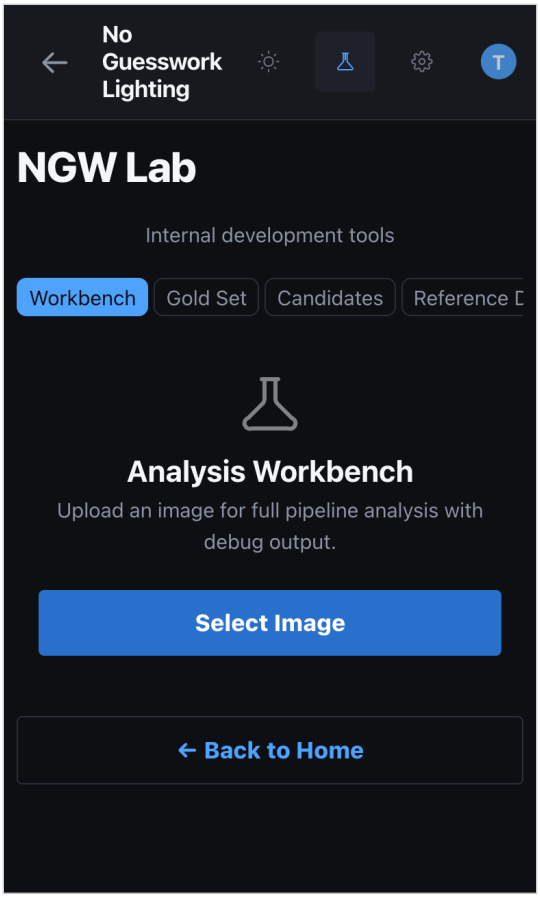
New reference images are added via API:

```
POST /api/lab/reference/ingest
```

Once ingested, entries appear as **"pending"** and can be reviewed. The UI does not have a direct upload button for the Reference Dataset — use the Workbench for one-off analysis, and the ingest API for curated dataset additions.

6 Common Workflows

Section 6: Common Workflows



Start every workflow from the Workbench — upload, analyze, then act

Workflow 1 — Testing a New Image End-to-End

1. Workbench → Select Image → Analyze
2. Review Formatted view: lighting family, modifier, light count
3. Switch to VLM vs CV: confirm signals agree or accept VLM overrides
4. Switch to Raw JSON: check `bestMatch.reliabilityScore` and `pattern_candidates`
5. If result is useful: Save to Gold Set
6. If the engine got something wrong: Propose Rule

Workflow 2 — Adding a Benchmark Image

1. Upload to Workbench, verify the analysis is correct
2. Save to Gold Set (status: Draft)

3. Edit the Gold Set entry to confirm expected_analysis matches ground truth
4. Set status to Approved
5. Run: `python3 scripts/run_benchmarks.py` to verify the entry passes

Workflow 3 — Investigating a SOFT_PASS Benchmark

1. Find the image path from benchmark output
2. Load it in Workbench with Debug Overlay enabled
3. Inspect the overlay: look for weak catchlights, occlusion shadows, ambiguous geometry
4. Check VLM vs CV tab for signal conflicts
5. If a rule fix is clear: Propose Rule with the relevant signal data attached

Workflow 4 — Reviewing Reference Dataset Images

1. Open Reference Dataset tab
2. Step through entries with ← / → navigation
3. Check Reference Analysis section: is the lighting family and setup correct?
4. If correct: Approve. If wrong framing, bad lighting, or ambiguous: Reject
5. If signals look stale after an engine update: Reprocess

7 Environment & Configuration

Section 7: Environment & Configuration

Environment Variables

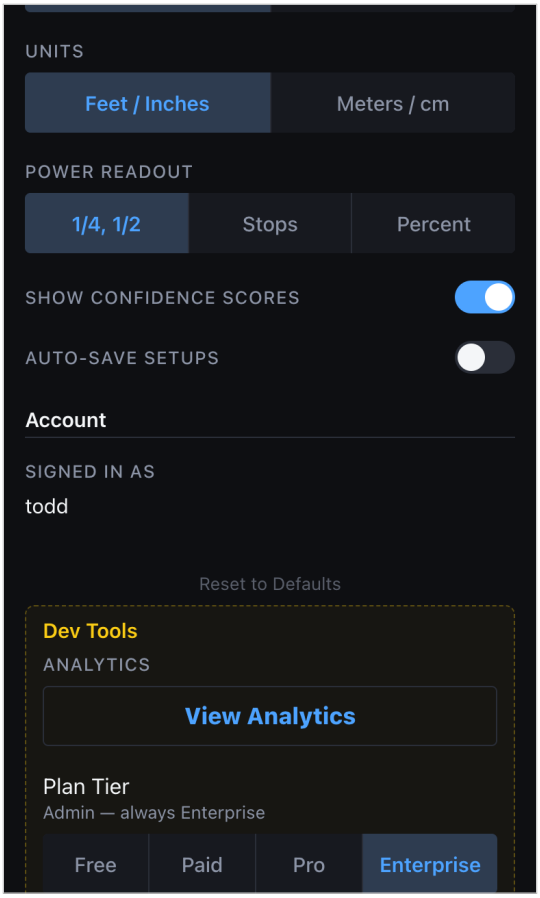
`VLM_PROVIDER`
Override auto-detection. Values: `openai` / `anthropic` / `none`. When unset or "auto", uses OpenAI if OPENAI_API_KEY is present, otherwise Anthropic, otherwise disables VLM.

`OPENAI_API_KEY`
Required when VLM_PROVIDER is "openai". Used for GPT-4.1 vision calls.

`ANTHROPIC_API_KEY`
Required when VLM_PROVIDER is "anthropic". Used for Claude Sonnet vision calls.

`VLM_MODEL`
Override the default model. Defaults: `gpt-4.1` (OpenAI), `claude-sonnet-4-20250514` (Anthropic).

Note: *VLM is optional. Without it, Workbench runs the full CV pipeline and returns results. The VLM vs CV tab is disabled and shows "VLM not configured" on hover.*



Dev Tools in Settings confirms your active plan tier and admin status

Rate Limit Handling (429 Errors)

The VLM layer automatically retries on rate-limit responses:

Attempt	Failure Action	Wait Before Retry
1	First call fails	2 seconds
2	Retry fails	5 seconds
3	Retry fails	15 seconds
4	Logs error; pipeline continues without VLM data	—

To request a rate limit increase: https://platform.openai.com/settings/organization/limits

Upload Limits

- Maximum file size: **10 MB**
- Supported formats: JPEG, PNG, WebP, HEIC, HEIF, TIFF

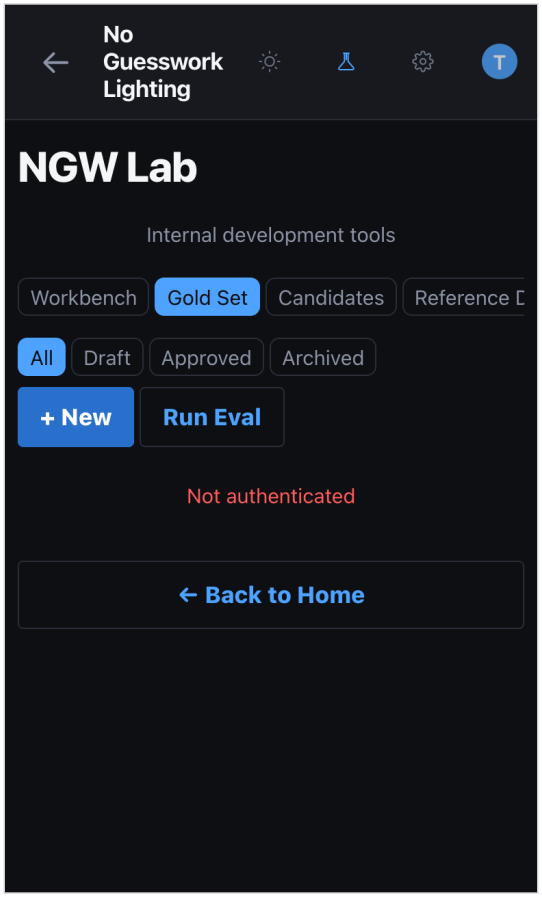
9 How Datasets Are Updated

Section 9: How Datasets Are Updated

The three datasets — Gold Set, Reference Dataset, and Candidates — are the learning substrate of the engine. Every time you add, approve, or reject an entry you are making a verifiable claim about what the correct output looks like for a given input. The engine is only as accurate as these datasets are correct.

Gold Set — Ground Truth for Benchmarks

The Gold Set is the evaluation benchmark. Each entry is an (image, expected_analysis) pair. When `run_benchmarks.py` runs, it calls `analyze_image()` on every approved entry and compares the live result against the stored `expected_analysis`. Entries that produce a matching pattern and confidence above threshold score **PASS**; partial matches score **SOFT_PASS**; failures score **FAIL**.

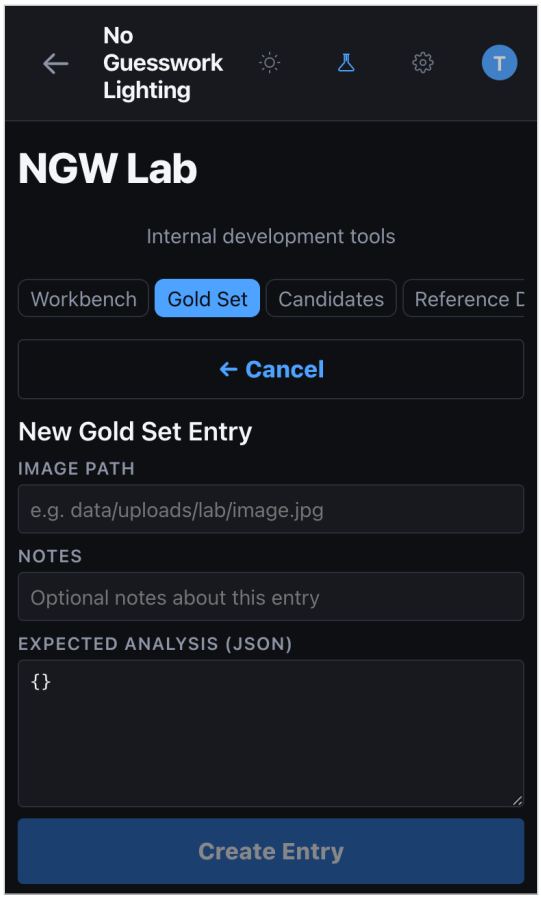


Gold Set showing Draft/Approved/Archived lifecycle filters

How an entry enters the Gold Set:

1. Upload the image in Workbench → run full analysis
2. Review Formatted and VLM vs CV views — confirm the output is correct
3. If VLM signals are better: accept overrides in VLM vs CV tab

4. Tap "Save to Gold Set" (green "✔ Commit to Gold Set" when VLM overrides were accepted)
5. In Gold Set tab: set status to **Approved** when you are certain the expected_analysis is correct
6. Run `python3 scripts/run_benchmarks.py` — the entry now affects the PASS/SOFT_PASS/FAIL score



The New Entry form — paste the expected_analysis JSON from Workbench

How entries evolve:

- **Draft** → first save from Workbench or manual entry. Not yet included in benchmark scoring.
- **Approved** → confirmed correct, included in benchmark runs. Requires human verification.
- **Archived** → retired from active benchmarks but preserved for historical reference.

Note: *Never approve a Gold Set entry unless you have verified the expected_analysis by hand. Incorrect ground truth produces misleading benchmark scores and corrupts the learning signal.*

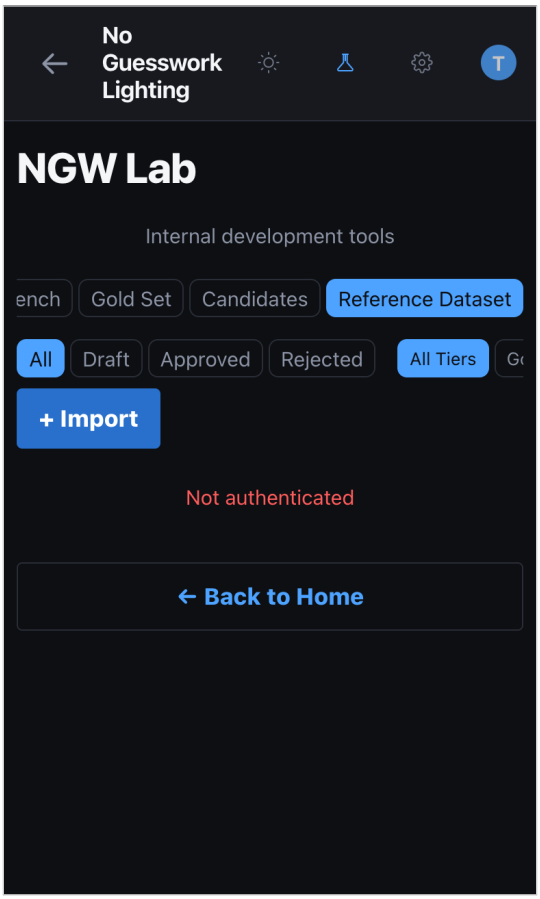
How to update an existing entry:

- Reopen the entry in the Gold Set tab
- Edit the expected_analysis JSON directly
- If the engine output has improved for this image, update expected_analysis to match the new correct output
- Save and re-run benchmarks to confirm PASS

Impact on engine: Benchmark score directly measures engine accuracy. Adding more approved Gold Set entries increases benchmark coverage. Every SOFT_PASS is a signal that a rule is close but needs refinement — create a Candidate to track the fix.

Reference Dataset — Curated Signal Library

The Reference Dataset stores curated reference photos with full pipeline analysis attached. It is not an evaluation benchmark — it is a signal library. The engine uses Reference Dataset entries as pattern exemplars during the shoot-match process: when a user uploads a photo, the engine compares it against approved reference entries to find the closest pattern match.



Reference Dataset — pending entries await approval before entering pattern matching

How an entry enters the Reference Dataset:

1. Ingest via API: `POST /api/lab/reference/ingest` with the image file
2. The pipeline runs automatically: CV passes + VLM signal extraction + VLM reconstruction
3. Entry appears in Reference Dataset tab as "pending"
4. Open the entry: review Reference Analysis section (lighting family, modifier, setup)
5. Check Pipeline Signals and VLM Reconstruction for consistency
6. If correct: Approve. If wrong or ambiguous: Reject.

7. Approved entries become active pattern exemplars

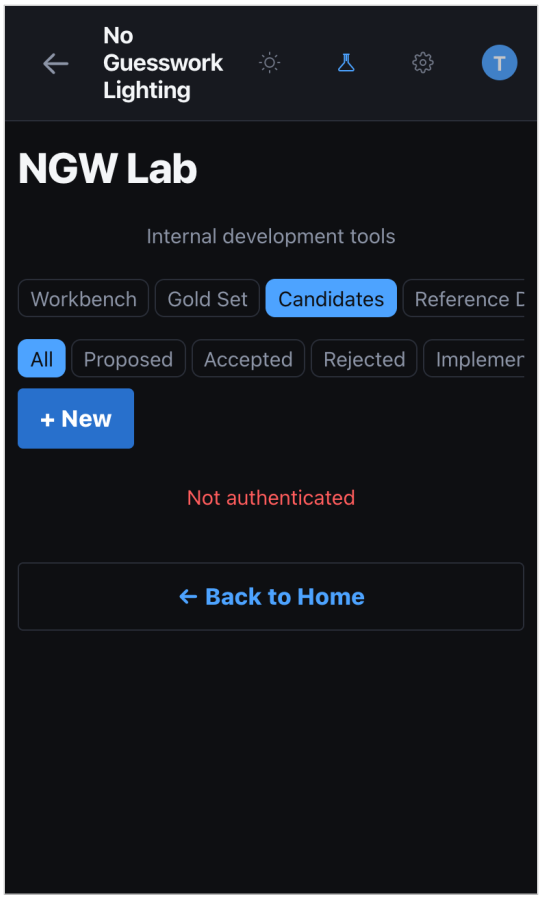
Keeping the dataset current:

- After an engine update, use **Reprocess** to re-run the pipeline on existing entries with fresh signals
- Reject entries where lighting is ambiguous, framing is poor, or the setup cannot be reliably identified
- Aim for 3–5 approved entries per lighting pattern family for robust coverage
- Approved entries with high VLM reconstruction confidence (>0.75) are the most valuable

Status	Included in Matching	Action
pending	No	Review in detail view — Approve or Reject
approved	Yes	Active exemplar — Reprocess after engine updates
rejected	No	Excluded — can be re-approved if issue is resolved

Candidates — Proposed Rule Changes

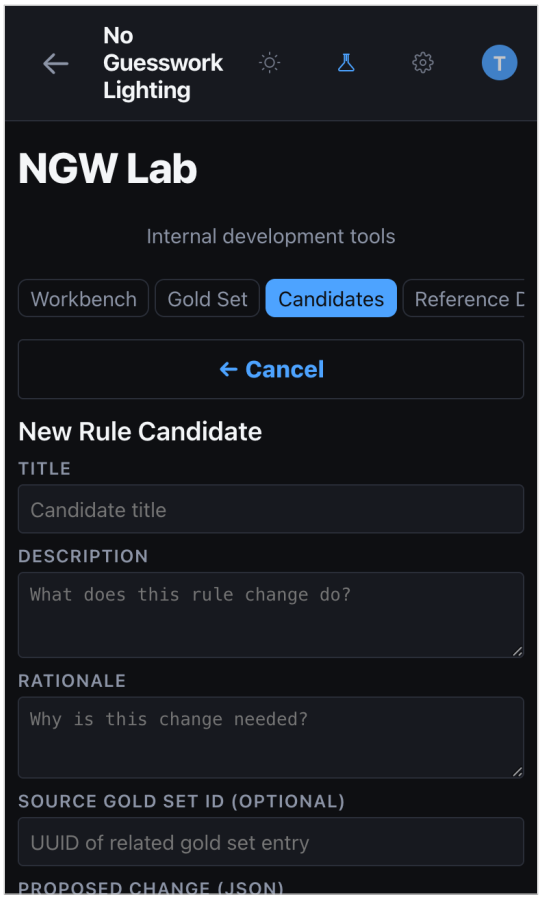
A Candidate is a formal proposal to change the engine's classification rules. Candidates are the change management system — they prevent ad-hoc rule edits and ensure every change is traceable to an observed failure.



Candidates tab tracking proposed rule changes by status

Lifecycle of a Candidate:

1. Observe a gap: Workbench shows wrong pattern, or benchmark shows SOFT_PASS on a clear case
2. Create candidate via "Propose Rule" in Workbench (auto-fills from analysis) or manually in Candidates tab
3. Set status to **Proposed** — record the failing image path, the incorrect output, and the expected output in the proposed_change JSON
4. Run benchmarks to establish baseline: document current PASS/SOFT_PASS/FAIL counts
5. Implement the rule change in the engine (pattern weights, cue thresholds, scoring adjustments)
6. Re-run benchmarks: confirm the target case improved without regressing others
7. Update Candidate status to **Accepted**. Add implementation notes.
8. Update any affected Gold Set entries if expected_analysis changed



New Rule Candidate — capture the failing pattern, evidence, and proposed fix

```
draft → proposed → review → accepted | rejected
```

***Note:** A Candidate should never be accepted until benchmarks confirm improvement with zero new FAIL regressions. If a rule change fixes one case but breaks another, split it into two Candidates.*

10 Lighting Intelligence — How the System Learns

Section 10: Lighting Intelligence — How the System Learns

Lighting Intelligence is the structured knowledge object that the engine produces for every analysis. It is not a static lookup — it is synthesised in real time from computer vision signals, VLM interpretation, environment inference, and pattern classification. Every field in `lightingIntelligence` reflects a specific inference step, and every field can be improved by better Gold Set coverage, better reference images, or a more precise classification rule.

What lightingIntelligence Contains

`lightingIntelligence` is assembled by `_build_lighting_intelligence()` in `engine/services/shoot_match_service.py`. It is the final output layer of the engine — the synthesised result after all CV passes, VLM calls, pattern classification, and environment inference have completed.

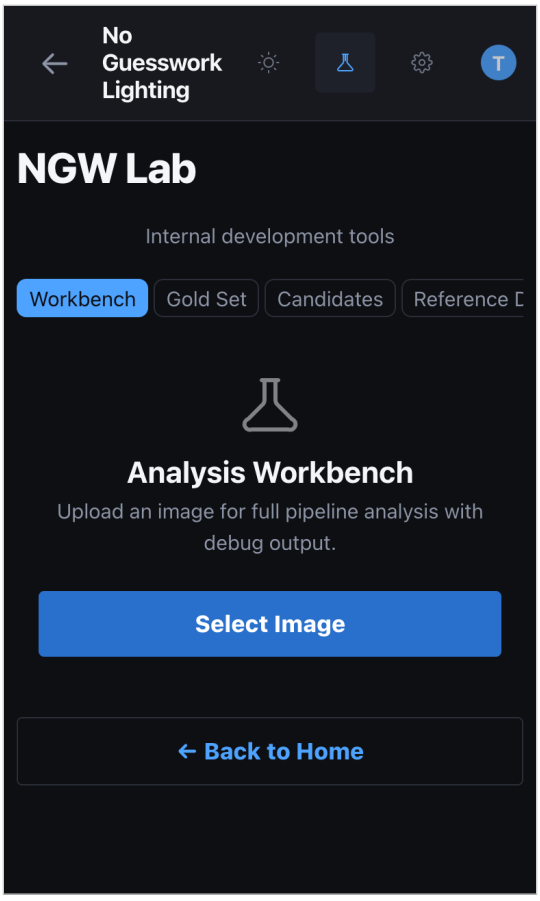
Field	Source	What It Means
`detectedPattern`	Pattern classifier + VLM reconciliation	The authoritative lighting pattern (rembrandt, butterfly, loop, split, clamshell, etc.)
`patternConfidence`	Reliability score from pattern candidates	How certain the engine is of the detected pattern (0.0–1.0)
`detectedModifier`	CV modifier shape solver + VLM reconstruction	The most likely light modifier (softbox, octa, beauty dish, umbrella, window, etc.)
`lightCount`	Light role hypothesis pass	Number of distinct light sources detected
`keyPosition`	Key light geometry inference	Position of the key light (e.g. "45° camera-left, high")
`fillMethod`	Fill role inference	How fill is achieved (reflector, second light, negative fill, none)
`lightSourceType`	Environment inference	natural / artificial / mixed / unknown
`ambientConditions`	Environment inference	Human-readable description of shooting conditions
`detectedCCT`	Color temperature pass	Estimated color temperature in Kelvin
`backgroundLight`	Background light role detector	Whether a dedicated background light is detected
`moodDiscrepancy`	User mood vs detected mood comparison	Flags when user's selected mood differs from what the image shows
`perceptionExplanation`	Perception layer (supporting/contradicting signals)	Supporting signals, contradicting signals, ambiguity flags

The Analysis Pipeline — How Each Signal Is Built

Every field in lightingIntelligence traces back through a specific pipeline stage. The engine runs these stages sequentially:

1. **CV Vision Pipeline** (`engine/vision_pipeline.py`) — MediaPipe face detection, shadow analysis, highlight mapping, catchlight geometry, surface classification, environment detection, color temperature. Produces raw signal measurements.
2. **VLM Signal Extraction** (`engine/vlm.py`) — A vision model (GPT-4.1 or Claude Sonnet) analyses the image and extracts observable physical signals: shadow vector, highlight width ratio, catchlight shape/position, head geometry, reconstruction estimates. Retries up to 4 times on 429 rate limits.

3. **VLM Reconstruction** (`engine/vlm_reconstruction.py`) — A second VLM call interprets the structured CV + VLM signals and produces a physical reconstruction: dominant source direction, source size class, modifier family candidates, light roles, environment type. This sits between raw signals and the rule engine.
4. **Cue Inference Pipeline** (`engine/cue_inference_pipeline.py`) — 23 typed cue fields are computed from all upstream signals. Each cue has a confidence score. Low-confidence and missing cues are flagged.
5. **Pattern Classification** (`engine/orchestrator.py`) — Pattern candidates are scored and ranked. The authoritative pattern is selected by reconciling the classifier result, VLM lighting_style, and reconstruction candidates. The reliability score measures signal consistency.
6. **Environment Intelligence** (`_describe_ambient_conditions`) — The environment inference result is translated into `lightSourceType`, `ambientConditions`, and `environmentConfidence`.
7. **Perception Layer** — Face validation, signal reliability assessment, edge-case flags (blown highlights, mixed color temperature, extreme low-key, B&W processing), and the perception explanation (supporting signals, contradicting signals, ambiguity flags).

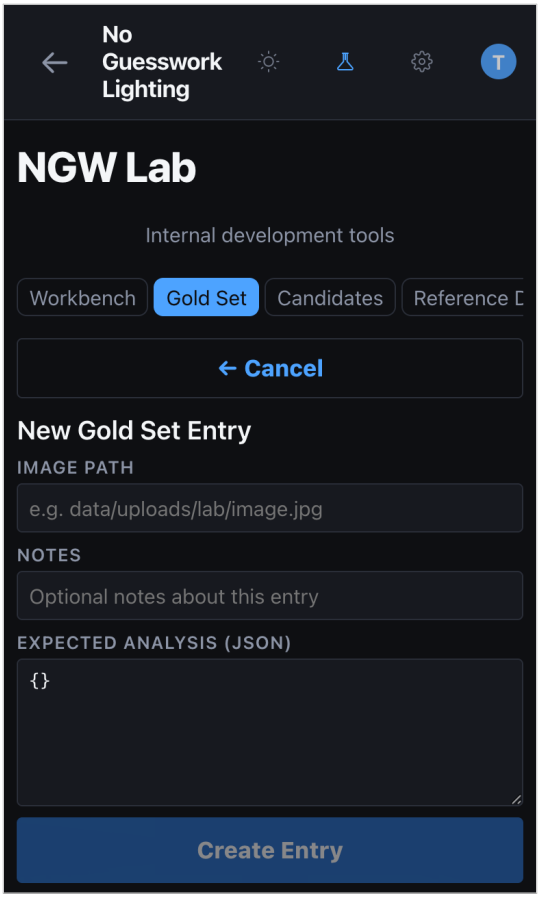


Every Workbench analysis runs the full 7-stage pipeline — results reflect all active signals

How the Engine Improves Over Time

The engine does not have a training loop — it improves through the deliberate human curation loop that the Lab enables. The learning cycle is:

1. **Observe** — An image produces an incorrect or low-confidence result. Found via: benchmark SOFT_PASS/FAIL, user feedback, Workbench testing.
2. **Diagnose** — Open the image in Workbench with Debug Overlay. Identify which signal is weak or wrong: catchlight geometry? shadow direction? VLM vs CV conflict? Check the perceptionExplanation ambiguity flags.
3. **Curate** — If the reference exemplars for this pattern are sparse, add more approved Reference Dataset entries for that lighting family. More exemplars → better pattern matching coverage.
4. **Propose** — Create a Candidate with the failing image, the incorrect output, the expected output, and the specific signal gap as evidence.
5. **Fix** — Implement the rule change: adjust pattern scoring weights, add a new cue threshold, refine the VLM reconstruction prompt, or update `_AMBIENT_DESCRIPTIONS`.
6. **Verify** — Re-run benchmarks. The target case must improve. No existing PASS should become FAIL.
7. **Commit** — Mark the Candidate as Accepted. Add the corrected image to the Gold Set. The next benchmark run reflects the improvement permanently.



End the improvement cycle by adding the verified result to the Gold Set as ground truth

***Note:** Every improvement to the engine should be traceable: failing image → Candidate → code change → benchmark delta → Gold Set entry. If any step in that chain is missing, the improvement cannot be verified or reproduced.*

What Makes a Signal "Better"

- **More Gold Set coverage** — Each approved entry adds a data point. Patterns with fewer than 3 Gold Set entries are under-tested. Aim for 5+ per pattern.
- **Higher VLM reconstruction confidence** — If `reconstruction_confidence` is below 0.6 for a pattern that should be clear, the VLM prompt or the signal aggregation needs refinement.
- **Fewer ambiguity flags** — The perception layer flags conditions that reduce reliability: `no_face`, `tiny_face`, `multiple_patterns_close_confidence`, `bw_limits_color_cues`, `low_signal_count`. Reducing these flags means the engine has more usable signal.
- **Tighter SOFT_PASS cluster** — As rules improve, SOFT_PASS cases should move to PASS. If SOFT_PASS count stays flat or grows, signals for those patterns are fundamentally weak and need more Reference Dataset entries or VLM prompt tuning.
- **Signal consistency (VLM vs CV agreement)** — In the Workbench VLM vs CV tab, disagreements between VLM and CV signals reveal where perception is uncertain. When VLM and CV agree, confidence is high. Systematic disagreements identify where one source is unreliable for a specific pattern.

11 FAQ

Section 11: FAQ

Q: I don't see the NGW Lab option anywhere in the app.

A: Lab is only visible to admin-whitelisted accounts. Confirm your login email is in the `ADMIN_EMAILS` list in `auth/dev_guard.py`. If it is and the tab still doesn't appear, check that the `enable_lab` feature flag is active in your environment.

Q: I see "Sign in required" when I open Lab.

A: You are not authenticated. Tap "Sign In" and log in with an admin email address. The Lab screen will appear after successful authentication.

Q: The VLM vs CV tab is greyed out after analysis.

A: Either no VLM API key is set in .env (add OPENAI_API_KEY or ANTHROPIC_API_KEY), or the VLM returned no data for this image. Check server logs for 429 rate-limit errors, timeout messages, or JSON parse failures.

Q: Analysis is taking a very long time or hanging.

A: The pipeline runs CV processing plus up to two VLM calls (signal extraction and reconstruction). With a rate-limited VLM, retries add 2 + 5 + 15 = 22 seconds before failing gracefully. If you want faster results without VLM, set `VLM_PROVIDER=none` in .env.

Q: I'm getting 429 rate limit errors from OpenAI.

A: The engine retries automatically (2s, 5s, 15s delays). If errors persist, you have hit your tier rate limit. Request an increase at https://platform.openai.com/settings/organization/limits or reduce benchmark concurrency.

Q: Images in the Reference Dataset tab show "Image unavailable".

A: All /api/lab/ image endpoints require a Bearer token. Images are fetched via authenticated blob requests automatically. If you see this error, the session may have expired — sign out and sign back in.

Q: I approved a Reference Dataset entry by mistake.

A: Currently there is no undo button. As a workaround, use the Reject action to re-mark the entry as rejected, or reprocess it to reset to pending status.

Q: How do I add an image to the Reference Dataset?

A: Reference images are ingested via the API: `POST /api/lab/reference/ingest`. Once ingested they appear in the grid as "pending" and can be reviewed. The UI does not have a direct upload button for the Reference Dataset.

Q: The Debug Overlay option isn't showing.

A: The Debug Overlay checkbox only appears before analysis runs (before tapping Analyze). If you have already run an analysis, tap "New Image" to reset and select the image again with Debug Overlay checked.

Q: A Gold Set entry shows as SOFT_PASS instead of PASS in benchmarks.

A: SOFT_PASS means the engine found the right lighting pattern but with lower confidence than expected, or a secondary signal differed from the expected analysis. Open the image in Workbench with Debug Overlay,

compare signals to your expected_analysis JSON in the Gold Set, and propose a rule change if the discrepancy is systematic.

Q: How do I run benchmarks?

A: From the server CLI: `python3 scripts/run_benchmarks.py`. The UI does not trigger benchmark runs. Results show PASS / SOFT_PASS / FAIL per entry with confidence scores and signal diagnostics.

Q: Can I export the Gold Set?

A: The Gold Set is stored in the database. You can query it directly or use the `/api/lab/gold-set` endpoint with your Bearer token to get all entries as JSON. There is no built-in export button in the current UI.

Q: What is the difference between Approve and Reprocess in the Reference Dataset?

A: Approve marks the entry as a valid curated reference (changes status, no pipeline re-run). Reprocess runs the full CV + VLM pipeline again on the existing image and updates the stored signals and analysis — useful after an engine update. Status is not changed by Reprocess.

Q: Can I use LAB on mobile?

A: Yes. The Lab screen is responsive and works on mobile. Image upload uses the native file picker. The Debug Overlay and JSON views are scrollable. For detailed signal review, a desktop or tablet is more comfortable due to screen width.